ME 371 - Data Driven Problem Solving

The Cooper Union for the Advancement of Science and Art

Fall 2025

Masoud Masoumi

Agenda

- Introduction to Machine Learning
- Running Example: Bearing Fault Detection
- Features and Measurements
- Training, Validation, and Testing
- **The Problem**: Feature Scaling
- **The Challenge**: Too Many Features
- **The Solution**: Principal Component Analysis (PCA)

Demo Code: We'll run Python code alongside these slides to see the concepts in action

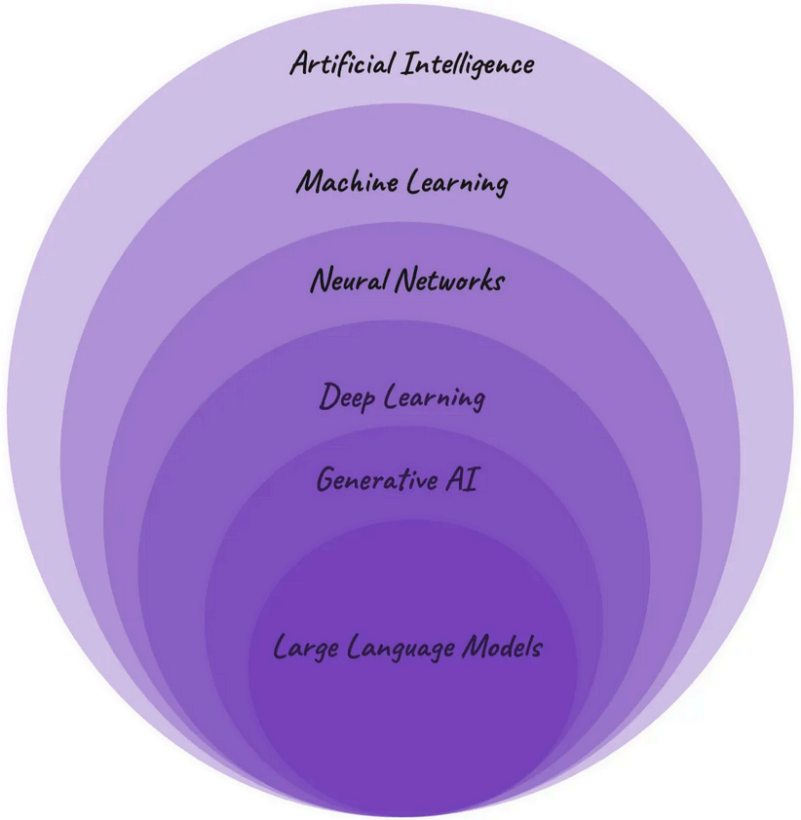
Introduction to Machine Learning

Machine learning enables systems to learn patterns from data.

For mechanical engineers:

- Predict when machinery will fail
- Optimize manufacturing processes
- Control quality in production
- Improve designs based on data
- Predict energy consumption

Today's Goal: Learn how to predict bearing failures using sensor data



(source for image)

Running Example: Bearing Fault Detection

The Scenario:

- A manufacturing plant has critical bearings in their machinery
- When a bearing fails unexpectedly: $100,000/hour downtime
- **Goal**: Predict failures before they happen using sensor data

The Data:

We have measurements from 500 bearings (250 healthy, 250 faulty):

- Vibration sensors measure shaking/movement
- Temperature sensors
- Measurements at different locations and directions

The Question: Can we use this data to predict which bearings will fail?

→ **Run Demo Code: Section 1 - Load and Explore Data**

Features: The Data We Measure

Features are the measurements we use as inputs for machine learning.

For our bearing example:

Feature Type	Examples	Typical Range
Temperature	Bearing surface temp	50 - 100°C
Vibration	How much it shakes	0.5 - 25 mm/s
Location	X, Y, Z directions	0.5 - 20 mm/s
Statistics	Average, maximum, variation	Various

Each bearing gives us about **12 measurements** = 12 features

Key Point: Faulty bearings vibrate more and run hotter than healthy ones

→ **Run Demo Code: Section 2 - Visualize Features**

Training, Validation, and Testing

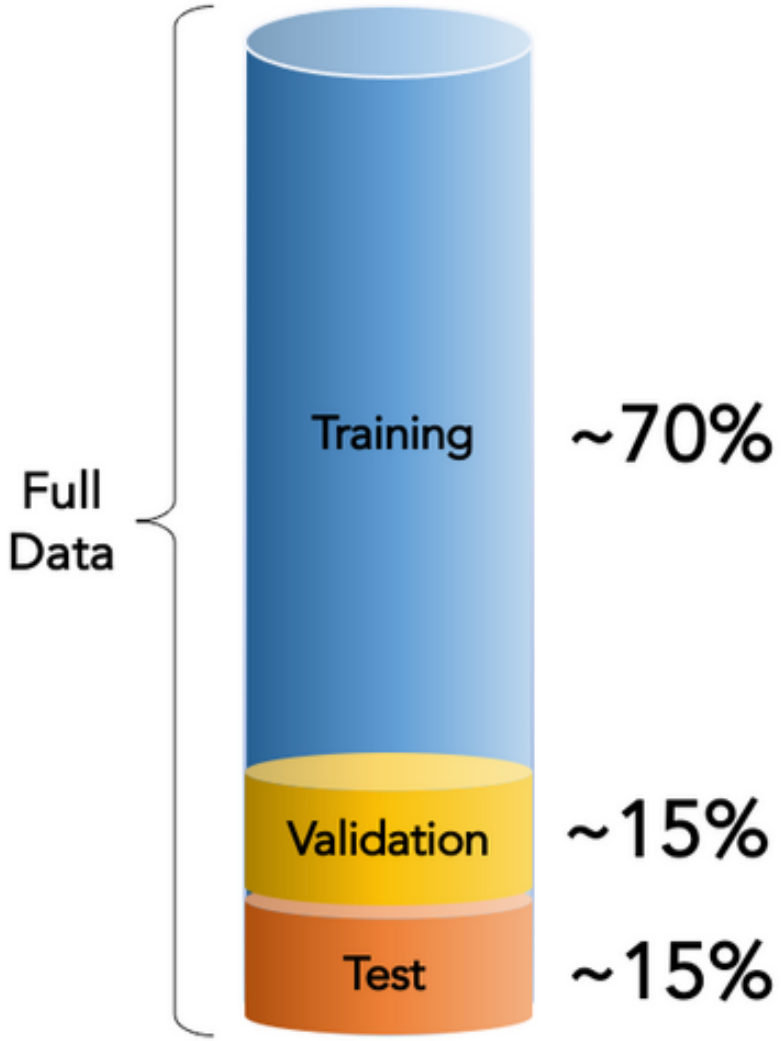
Like learning for an exam:

- **Study** examples (Training)
- **Practice** problems (Validation)
- **Take** the exam (Testing)

For our bearings:

1. **Training (70%)**: 350 bearings
   - Algorithm learns patterns
   - "High vibration + high temp = likely faulty"
2. **Validation (15%)**: 75 bearings
   - Check if learning is working
3. **Test (15%)**: 75 bearings
   - Final check on completely new bearings

→ **Run Demo Code: Section 3 - Split the Data** (Here, we're keeping it simple with just train/test.)



(source for image)

The Problem: Features Have Different Scales

Look at these three measurements from the same bearing:

```
Temperature:  75.3 °C
Vibration:    12.8 mm/s
Imbalance:    0.043 mm
```

The Issue:

- Temperature values: 50-100 (range of 50)
- Vibration values: 0-25 (range of 25)
- Imbalance values: 0.01-0.15 (range of 0.14)

Problem for ML: The algorithm "thinks" temperature is more important just because the numbers are bigger!

Solution: Scale all features to similar ranges before using them

Feature Scaling: Two Common Methods

1. Min-Max Normalization - Scale to range [0, 1]

- Formula: new_value = (value - minimum) / (maximum - minimum)
- Example: Temperature 75°C → 0.5 (if range is 50-100°C)
- Use when: You know the physical limits

2. Standardization - Transform to mean=0, std=1

- Formula: new_value = (value - average) / standard_deviation
- Example: If average temp = 70°C, std = 10°C, then 75°C → 0.5
- Use when: Data has outliers or unknown limits

After scaling: All features are on comparable scales

- Fair comparison between measurements
- Better ML algorithm performance

→ **Run Demo Code: Section 4 - Apply Feature Scaling**

Visualizing the Impact of Scaling

Before Scaling:

- Temperature dominates (50-100)
- Imbalance barely visible (0.01-0.15)
- Unfair comparison

After Scaling:

- All features comparable
- Each contributes fairly
- Meaningful distances

Think of it like: Converting all units to the same scale before comparing (like converting everything to meters before measuring)

Example Distance Calculation:

Without scaling:

- Difference is dominated by temperature

With scaling:

- All features contribute proportionally
- Distance reflects true similarity

→ **Run Demo Code: Section 5 - Compare Scaled vs Unscaled**

The Challenge: Too Many Features - "Curse of Dimensionality"

Our current bearing monitoring:

- 12 features from basic sensors
- Temperature, vibration (X,Y,Z), speed, load, etc.

What if we expand monitoring in a real facility?

- 10 sensor locations × 3 directions = 30 vibration measurements
- Temperature at each location = 10 more features
- Multiple measurement frequencies = 30 more features
- Statistical calculations (max, min, average, range) = 40 more
- **Total: 100+ features easily!**

Problems with many features:

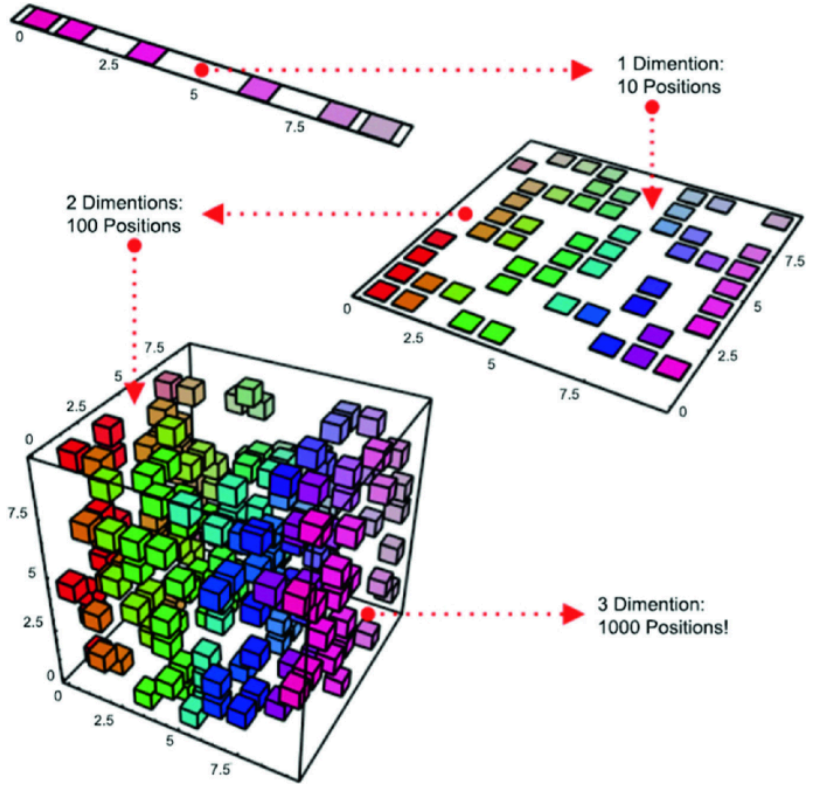
1. **Computational**: Training takes much longer, needs lots of memory
2. **Data Hungry**: Need exponentially more samples
3. **Overfitting**: Too many ways to find false patterns
4. **Redundancy**: Many features measure similar things

The Curse of Dimensionality: Intuition

1D Example: 10 points cover a line nicely
2D Example: Need 100 points (10×10) to cover a square
3D Example: Need 1,000 points (10×10×10) to fill a cube
12D Example (our bearings): Would need 10^12 points!
The Problem: Data becomes very sparse in high dimensions

Real Impact:

- Need massive amounts of data
- Training becomes very slow
- Risk finding patterns that aren't real



(source)

The Solution: Principal Component Analysis (PCA)

Key Insight: Many features are related to each other!

- If vibration is high in X direction, it's probably high in Y too
- If one temperature sensor is high, nearby ones are likely high
- We're measuring the same underlying problem in multiple ways

What PCA Does:

- Finds the most important patterns in the data
- Combines correlated features into single "components"
- For example it might reduces 100 features → 10 components while keeping most information

Example: Instead of tracking X, Y, Z vibration separately, PCA might create:

- Component 1: "Overall vibration intensity"
- Component 2: "Directional imbalance"
- Component 3: "Temperature effect"

PCA: Finding New Directions

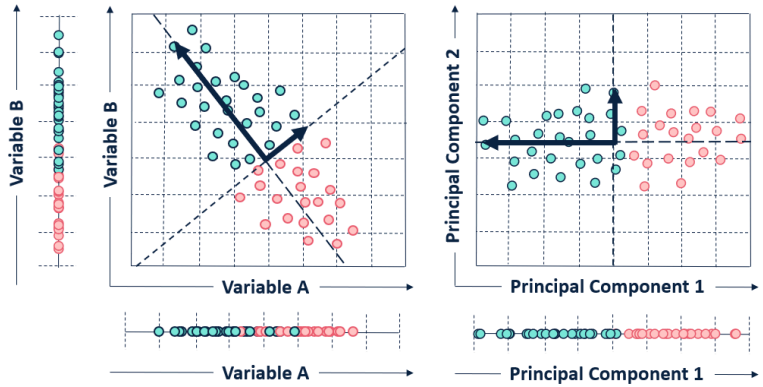
Imagine looking at scattered data points:

- PCA finds the "best" direction to view them
- **First direction (PC1)**: Points spread out the most
- **Second direction (PC2)**: Next most spread, perpendicular to PC1
- Continue for as many components as needed

Think of it as:

Rotating your view to see the data from the most informative angles

→ **Run Demo Code: Section 6 - Apply PCA**



Red arrows = Principal Components

Points = Our bearing data

- See this page for more rigerous mathemaical explanations or watch the first 4 videos from this youtube playlist.

How Much Information Do We Keep?

PCA Results for Our Bearings:

Here is a possible scenario for information capturing PC components (imaginary scenario)

Components	Info Captured	What This Means
First 1	38%	One number captures 38% of variation
First 2	58%	Two numbers capture 58%
First 3	72%	Three numbers capture 72%
First 5	85%	Five numbers capture 85%
First 8	95%	Eight numbers capture 95%

Decision: Usually keep components that capture 90-95% of information

Result: 12 features → ? components (still useful, but simpler!)

→ **Run Demo Code: Section 7 - Analyze Explained Variance**

Visualizing the Data After PCA

After PCA, we can plot our bearings in the new component space:

Benefits of visualization:

- See if healthy and faulty bearings separate clearly
- Understand the most important patterns
- Check if our features are useful

Interpretation:

- **PC1 (horizontal)**: Likely represents overall bearing health
  - Left = healthy, right = faulty
- **PC2 (vertical)**: Might represent specific failure modes
  - Different types of problems

→ **Run Demo Code: Section 8 - Visualize PCA Results**

What Do Components Mean Physically?

PCA tells us which original features contribute to each component:

Example Component 1 might combine:

- 40% from X-vibration
- 35% from Y-vibration
- 30% from Z-vibration
- 20% from temperature
- → Represents "Overall bearing condition"

Component 2 might combine:

- 60% from temperature
- -30% from average vibration
- → Represents "Thermal problems vs mechanical problems"

Key Point: Components are weighted combinations of original measurements that capture the most important patterns

→ **Run Demo Code: Section 9 - Interpret Component Weights**

PCA and Linear Algebra Connection (Optional)

PCA is computed using a technique called **Singular Value Decomposition (SVD)**

Simple explanation:

- Any data matrix can be broken down into three simpler matrices
- Written as: **X = U Σ V^T** (See this page)
- The V matrix contains our principal component directions
- The Σ values tell us how important each component is

Why use SVD?

- More numerically stable than other methods
- Works efficiently even with many features
- Gives us all the information we need for PCA

NOTE: For a thorough mathematical explanations of SVD and various exmaple os its applications for dimensionality reduction/data compression, see "Data-driven science and engineering: Machine learning, dynamical systems, and control" - Part I - Dimensionality Reduction and Transformation.

Putting It All Together: The Complete Pipeline

Our journey from raw data to predictions:

1. **Collect Data**: 500 bearings with 12 measurements each
2. **Split Data**: 70% train, 15% validation, 15% test (in this simple case, just 30% test)
3. **Scale Features**: Standardize all measurements (mean=0, std=1)
4. **Apply PCA**: Reduce from 12 features to less number components (95% info kept)
5. **Train Model**: Use the reduced components to predict bearing health
6. **Evaluate**: Check accuracy on test set

Why each step matters:

- Scaling → Ensures fair feature comparison
- PCA → Reduces complexity while keeping information
- Result → Faster, more robust predictions

→ Run Demo Code: Section 10 - Complete Pipeline

Results: Before and After PCA

Comparison on our bearing dataset:

Key Takeaways:

- Lost very small amount of information
- Same prediction accuracy
- Faster training
- More interpretable (can visualize in 2D/3D)

Trade-off: Slightly less information, but much more practical

Real-World Applications for Mechanical Engineers

1. **Predictive Maintenance**:
   - Monitor machinery vibration, temperature, pressure
   - Reduce hundreds of sensor readings to key health indicators
2. **Quality Control**:
   - Track many dimensions of manufactured parts
   - Identify key factors affecting quality
3. **Design Optimization**:
   - Explore many design parameters
   - Focus on the most influential combinations
4. **Energy Systems**:
   - Monitor building performance with many sensors
   - Identify primary factors affecting efficiency

Common pattern: Lots of measurements → PCA → Simplified understanding

Key Takeaways

1. **Feature Scaling** is essential before analysis
   - Different units/scales can mislead algorithms
   - Always scale before applying PCA or ML
2. **Too many features** causes problems
   - Need more data, slower training, risk of overfitting
   - Called the "curse of dimensionality"
3. **PCA reduces dimensionality** intelligently
   - Combines correlated features
   - Keeps most important information (typically 90-95%)
   - Makes data easier to work with and visualize
4. **The pipeline matters**:
   - Scale → Reduce dimensions → Train model
   - Each step builds on the previous one